there's little reason to look back. In this section I discuss some of the cases in which you might still want to use the generic host approach.

In three main cases, you'll likely want to stick with the generic host instead of using minimal hosting with `WebApplication`:

- When you already have an ASP.NET Core application that uses the generic host
- When you need (or want) fine control of building the `IHost` object
- When you're creating a non-HTTP application

The first use case is relatively obvious: if you already have an ASP.NET Core app that uses the generic host and `Startup`, you don't need to change it. You can still upgrade your app to .NET 7, and you shouldn't need to change any of your startup code. The generic host and `Startup` are fully supported in .NET 7, but they're not the default experience.

TIP In many cases, upgrading an existing project to .NET 7 simply requires updating the framework in the .csproj file and updating some NuGet packages. If you're unlucky, you may find that some APIs have changed. Microsoft publishes upgrade guides for each major version release, so it's worth reading these before upgrading your apps: http://mng.bz/zXX1.

If you're creating a new app, but for some reason you don't like the default options used by `WebApplicationBuilder`, using the generic host may be your best option. I generally wouldn't advise this approach, as it will likely require more maintenance than using `WebApplication`, but it does give you complete control of your bootstrap code if you need or want it.